

当团队 90% 以上的代码由 AI 生成，31 万行的复杂业务系统还在高速膨胀，你会发现一个反直觉的事实：AI Coding 不会自动收敛复杂度 —— 没有统一规范的约束，不同人用 AI 写出的代码风格各异，系统反而会加速腐化。

本文记录了我们如何在不停止业务交付的前提下，完成这场重构。在这个过程中，我们积累了三个关键经验，希望这篇实战经验能提供一些可复用的思路。

- **经验一：**用Agent评测思路管理AI Coding。我们团队负责 Agent 评测业务，在实践中沉淀出一套核心标准对齐理念：“人人对齐→人机对齐”。我们发现管理 AI Coding 的底层逻辑一模一样 —— 先让团队形成统一共识（人人对齐），再将共识固化为 AI 可执行的约束（人机对齐）。本质上，就是同一套方法论在两个领域的复用。
- **经验二：**AI 正在重新定义“经验”的价值边界。利用 AI 工具，工程师短时间内就发现了 10 个性能隐患——过去需要长期积累才能建立的代码全局感，现在借助 AI，团队中的每个人都能快速具备。经验的价值正在从“能看全”转移到“能判断什么重要”。
- **经验三：**技术债可以像业务需求一样被迭代消化。行业谈重构，要么推倒重来，要么申请专项。我们给出了第三条路：把技术债拆解为业务需求的“顺带动作”，借着迭代渐进式消化。

一、背景

Agent评测系统长期承载多个核心业务场景，它同时承担了数据生产、流程编排、质量控制与多人协作等复杂能力，业务复杂度和工程复杂度都很高。具体来看，我们面对的复杂性主要体现在三个维度：

- **业务仍处于探索期，导致需求高度模糊：**全行业都在探索 Agent 评测，用户也不了解应该如何评测。这个大背景导致评测的需求又急又模糊。急，希望快速试错；模糊，业务方也不确定这条路是否真的有价值。
- **庞大且高频的迭代体量：**系统从 2025 年 6 月约不足 5 万行代码快速扩展至 31 万行，保持着月均 16 个需求（80% 业务需求 + 20% 技术需求）的高负荷运转。
- **“笛卡尔积”级别的业务场景矩阵：**系统底层支持 6 种多模态数据评测，上层构建了多种核心任务视图和精细化业务动作，并配套了十余种质检机制。这些能力交织着多种标签体系与动态分配策略，意味着系统每天都需要稳健处理成百上千种截然不同的复杂业务流组合。

二、为什么要重构？

当业务进入快速迭代与试错期，上述庞大的业务体量与原有底层架构之间的矛盾就会集中爆发，迫使我们必须启动本次大规模重构。核心动因直指以下三个痛点：

1. 业务模型亟需升级，旧架构无法支撑探索性业务

随着业务交互的丰富度和复杂度增加，旧有数据模型扩展能力不足导致“烟囱式”功能开发，几乎每新增业务形式都需要新增代码来实现。

2. 代码严重腐化，技术债拖垮迭代效率

过去长期采用“按需求建包”的模式开发，代码缺乏合理的工程分层，Controller 等各种复杂逻辑揉在一个包内，形成了严重的“面条式代码”。在 31 万行代码的体量下，这种深度的技术债让日常开发“牵一发而动全身”，导致一线同学开发异常痛苦，交付效率遭遇严重瓶颈。

3. 协作模式风险放大，缺乏规范的 AI Coding 加速系统腐化

一年左右的时间，团队成员规模增至 3 倍，并且团队成员技术背景复杂，涵盖高并发、机器学习离线训练、管理后端开发以及实习生，复杂业务系统开发经验不足。在这样一个高人员流动和跨技术栈的背景下，再叠加 90% 以上代码由 AI 辅助编写这一事实，如果不建立硬性的底层架构规范，不同背景的同学各自用 AI Coding，系统必将以极快的速度产生不可控的腐化与新债。

因此，我们不仅需要工程重构，而且要建设符合 AI Coding 规范的工程重构。规范才可以帮助我们团队消灭旧技术债，规避新技术债。

三、重构时间线与执行路径

阶段一：定义问题，借助 AI 梳理技术债（2026 年2月启动）

在需求高压背景下，要梳理技术债面临着一个极其现实的困境：量太大，根本看不完，也看不全。

面对膨胀至 31 万行以上的代码库，试图靠人力逐行阅读来建立全局的可靠认知是不现实的。我们的代码库中同样伴随着典型的高危特征：很多地方文档不全、大量隐式逻辑和历史兼容分支藏在细节里。一个看起来不起眼的接口，背后可能挂着一串极长的调用链。所以，梳理技术债最大的难点，在于人力永远无法在短时间内穷举和穿透这些错综复杂的关联逻辑——单段代码谁都能读懂，但没人能在短时间内把 31 万行的调用链全部穿透。

我们采用的是一种更适合复杂系统的方式：“专家经验定向 + AI 辅助排查”。

不再试图人工遍历，而是由核心开发圈定高危的排查边界，然后把穷举和扫描的脏活累活交给 AI。通过这种方式，我们快速摸清了系统底层的 P0/P1 级技术债（如业务模型缺陷、数据库查询性能隐患、状态管理技术债、索引技术债等）。

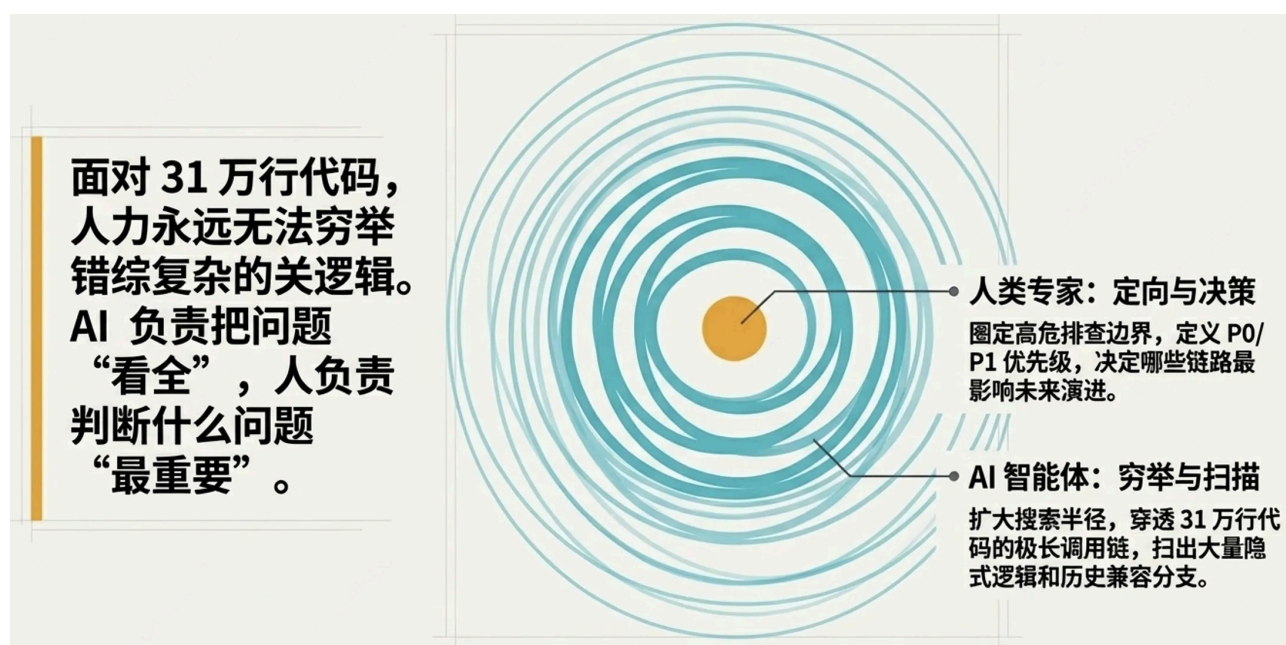
这一步中，我们最大的体会是 AI 很适合帮我们把问题“看全”，但什么问题最重要，什么问题值得优先改，还是要由人来判断。具体来说，人负责圈定 P0/P1 级问题和优先级，AI 负责在圈定的方向上做穷举扫描——比如梳理业务模型问题、定位大数据量性能隐患、排查状态管理和索引层面的技术债。

实践下来，这一步的 ROI 很高。我们仅仅投入了有限的资源，就完成了 3 个 P0 技术债和 2 个 P1 技术债的梳理。但最让我们意外的是下面这件事：

短时间内，工程师就利用 AI 辅助精准定位了 10 个隐藏极深、靠肉眼极难发现的性能隐患。这些隐患藏在复杂的调用链深处，即使是资深工程师逐行阅读也很难穷举到。这在纯人工阅读代码的模式下是几乎不可能的。

这个结果迫使我们重新思考“经验”的定义。过去，“能看全”是资深工程师的核心壁垒——你需要在系统里泡三年，才能建立起对调用链、隐式依赖和历史兼容逻辑的全局感知。但 AI 把“看全”的门槛打到了几乎为零。经验的价值正在从“能看全”转移到“能判断什么重要”——这才是人不可替代的部分。

这一步对我们后面的启发很大，因为只有问题定义清楚了，后面的规范、分层和迁移，才不会做成无源之水。



阶段二：调研并制定 AI 友好的研发规范（2026年2月底完成）

通过技术债梳理，我们解决了重构哪里的问题，那么接下来要解决的就是“代码应该怎么写”。在全员 90% 代码依赖 AI Coding 的现状下，核心要解决的问题是“如何将一两个用好 AI 的人的经验，高质量泛化到全组”。

为什么规范的价值被放大了？

在传统研发模式下，开发规范的主要作用是帮助团队协作、Code Review 和新人上手。但当 AI 已经成为主要编码产能后，规范的意义发生了本质变化。大模型生成代码时，会强依赖当前上下文和现有代码模式。如果代码库本身风格混乱、团队对规范理解不一致，AI 不会自动纠偏，反而会把差异进一步放

大，导致多人协作下持续产出”千人千面”的代码。因此，AI Coding 时代的研发规范已经升级为约束 AI 产出、阻止系统继续长新债的基础设施，远不止协作建议那么简单。

用评测 Agent 的方式，管理 AI Coding

但只让 AI 遵循规范还不够 —— AI 只能执行输入，不能替代团队形成统一判断。如果团队成员自己没有先对齐分层原则、建模方式和依赖边界，同一份规范就会被不同人解释成不同版本。

这个问题让我们想到了自己的本职工作。我们团队负责 Agent 评测业务，在长期实践中沉淀出一套核心理念：

- 标准对齐（人人对齐）：需要 1 位强有力的角色拉齐产品、运营、算法、QA 等所有角色的评测标准 —— 1 个”独裁者”好过 10 个”民主者”。
- 人机对齐：评测标准对齐后，通过模型选型和评测指标的优化，实现人机对齐，人机一致率达到基本阈值（例如 90%），才能认为机器的评价可信。

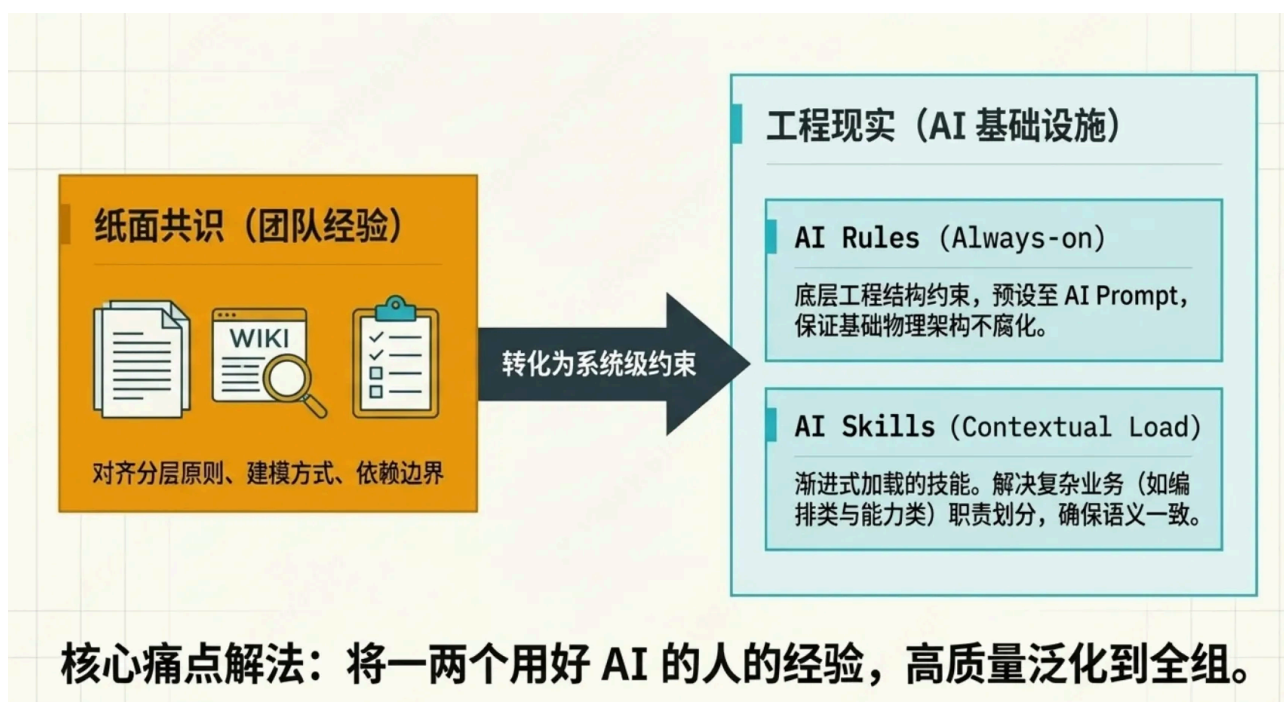
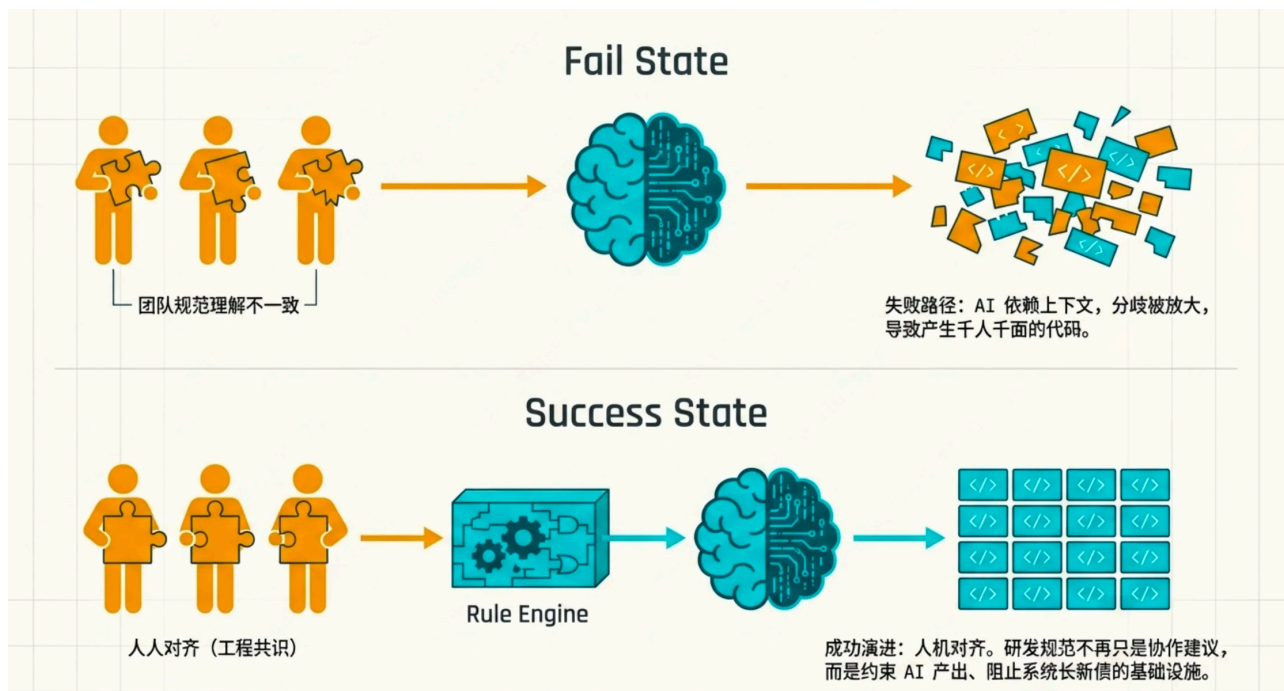
我们发现，管理 AI Coding 与评测 Agent 的底层逻辑一模一样。先通过规范拉齐团队的工程标准（人人对齐），再通过 AI Rule 和 Skill 约束大模型的生成结果（人机对齐）。一个做 AI 评测的团队，用评测的思维解决了工程治理问题。

顺序至关重要：先”人人对齐”，再”人机对齐”。很多团队以为配置好 AI Rule 就完事了，但真正的瓶颈在人，不在工具。团队自己没有统一共识，AI Rule 写得再好也会被不同人解释成不同版本。人的共识是 AI 约束的前提。

将规范转化为 AI 的执行约束

我们先调研了业内成熟团队的研发规范，并结合自身流程，沉淀出一套 AI 友好的工程约束，包括工程分层规范、业务域模型规约和仓储层规约。关键一步是没有把规范停留在文档层面，而是将其落地为 always 级别的 AI Rule，用于约束 AI 编码过程，并前置到预 CR 环节，帮助研发在提交前完成基础规范校验。

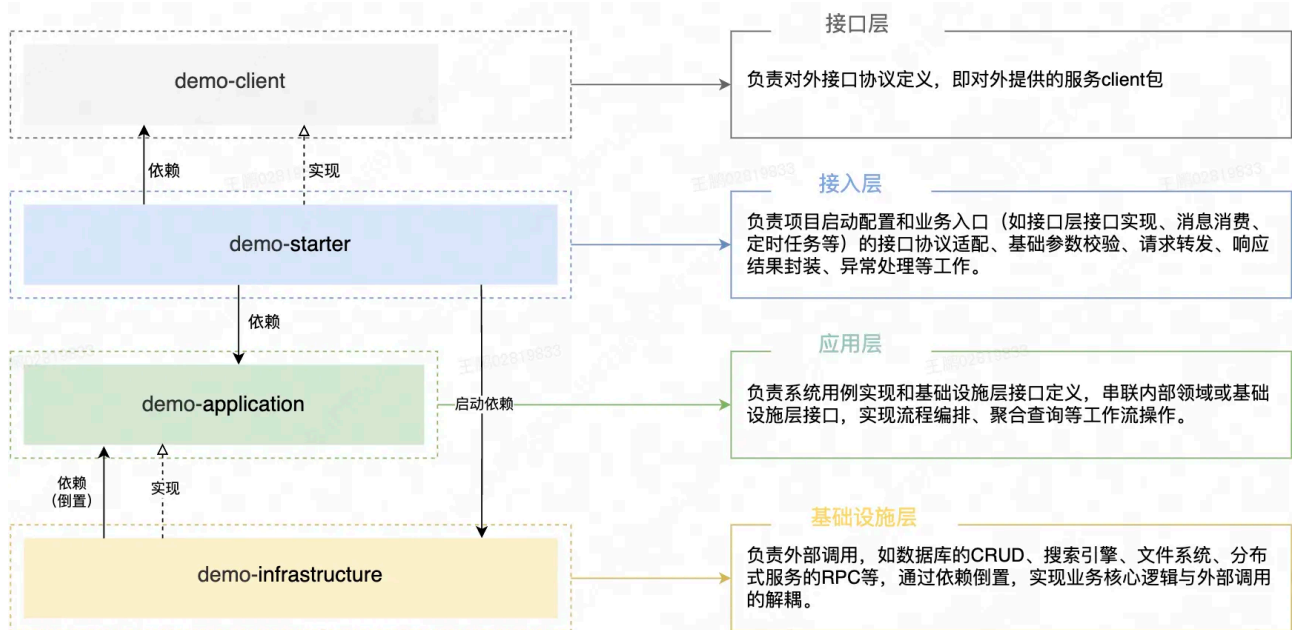
与此同时，针对最容易产生分歧的领域职责划分问题，我们围绕”编排类”与”能力类”的职责边界进行了组内统一，并将共识沉淀为编码时渐进式加载的 Skill。



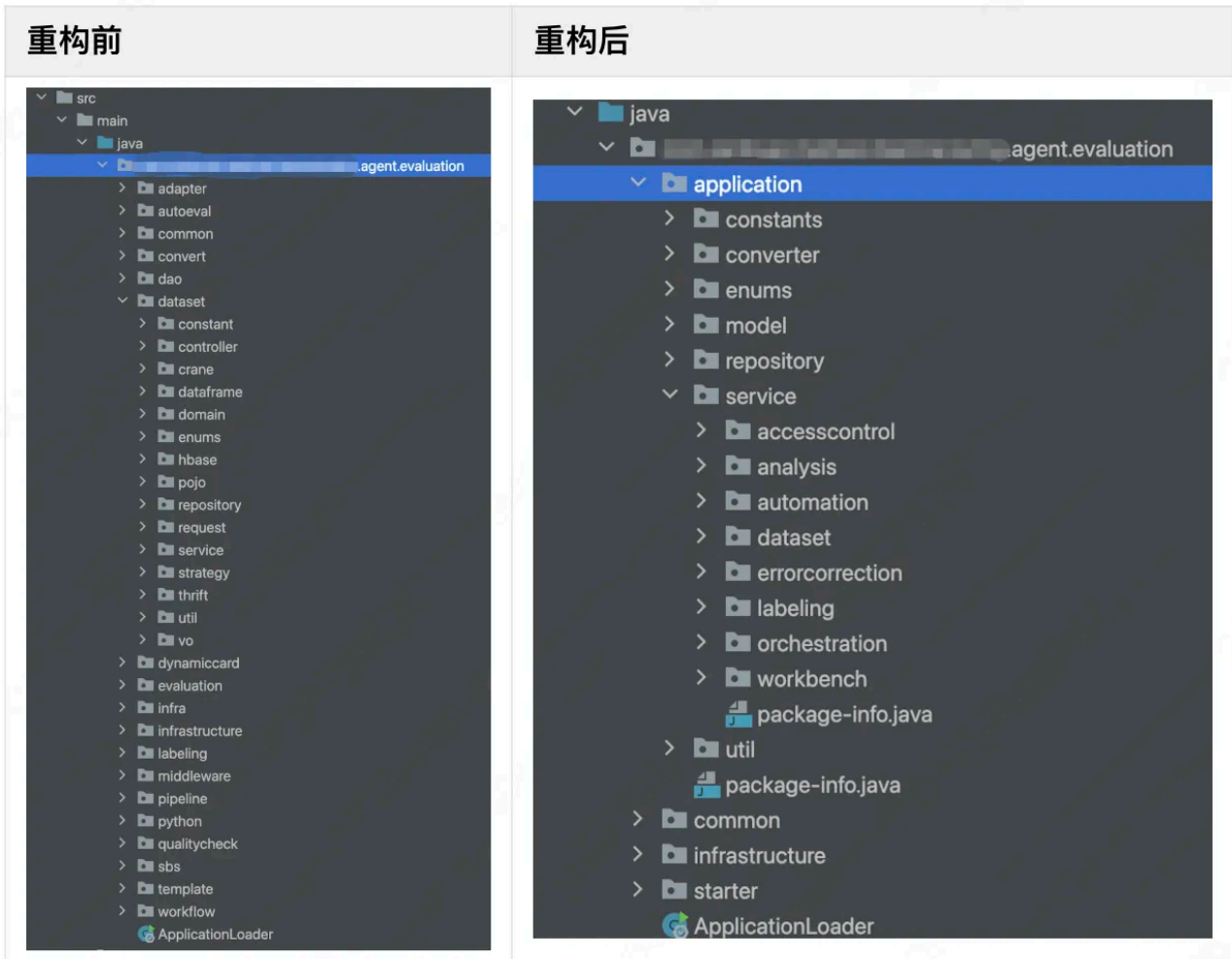
阶段三：建立 SOP，“见缝插针”完成渐进式重构（2026年3月– 4月）

Action 1: 100% 借助 AI 完成工程分层与解耦重构

我们将过去“按需求建包”的面条式代码，逐步迁移到标准四层架构（Starter / Application / Infrastructure / Common）以及按业务域组织的新结构中。但这次重构的重点，并不只是物理目录的调整，而是借此机会系统性治理历史代码中长期存在的深度耦合问题，尤其是底层数据对象 PO 在全链路中的泄露与上浮。围绕这一问题，我们分三步推进：第一步，补齐业务对象与数据转换层，收口散落各处的转换逻辑；第二步，在 Application 层重建接口契约，严格阻断底层数据对象向上层泄露；第三步，基于新契约修复上游全链路的参数依赖。



这类重构的特点是：改造规则相对明确，但涉及范围极广、重复劳动密集。我们的做法是先由重构主 R 亲自完成两个最复杂包的迁移，在过程中沉淀出一套可让 AI 执行的标准化迁移 SOP。有了这套 SOP，重构工作不再依赖某一个人的经验——团队其他成员只需按照 SOP 指导 AI 完成剩余包的迁移，研发本人聚焦业务语义验收和 Code Review 即可。通过这种“主 R 打样 → SOP 分发 → 全组并行执行”的方式，我们快速完成了十余个核心包的工程结构迁移。



Action 2: 零排期重构——借着业务需求“渐进式重构业务模型”

本次重构的深水区。行业里谈重构，通常只有两条路：要么推倒重来，要么申请专项排期。我们走了第三条路——把技术债拆解为业务需求的“顺带动作”，借着迭代渐进式消化，没有申请一天专门的重构时间。

具体做法是将技术债拆解到日常高优需求中。例如，借着某个核心功能迭代需求，顺势设计并落地了全新的业务模型；借着另一个功能升级需求，我们设计了全新的质检业务模型，并在 3 月下旬完成了全量迁移（一举兼容了多条业务链路，以及多视图、多区域的复杂交叉验证）。

这条路的难点在于拆解的精度——哪些业务需求能“顺带”消化哪些技术债，需要逐个判断：既不能让重构拖慢业务交付，也不能让业务需求绕过技术债继续堆新债。最终我在不停止业务交付的前提下，完成了核心数据模型的平滑升级。

Action 3: 重构质量保证

1. 建设 AI CR 与 Pre-PR 机制

随着 AI 编码效率飞跃式提升，我们很快遇到了“木桶效应”：Code Review 成了全链路中最拥堵的瓶颈：AI 极大地压缩了编码时间，压力系统性地向下游 CR 环节集中。如果 CR 效率不提升，AI Coding

的提效红利会被 CR 瓶颈吞掉。

我们团队达成的共识：

- 人工CR的价值，应该从“你写得对吗？”转变为“我们是否在正确的约束下解决正确的问题？”
- AI 审查规范类问题，做业务逻辑初筛；
- 人重点在前置技术方案评审环节把关，Review 最终代码实现是否符合技术方案、代码业务逻辑问题。

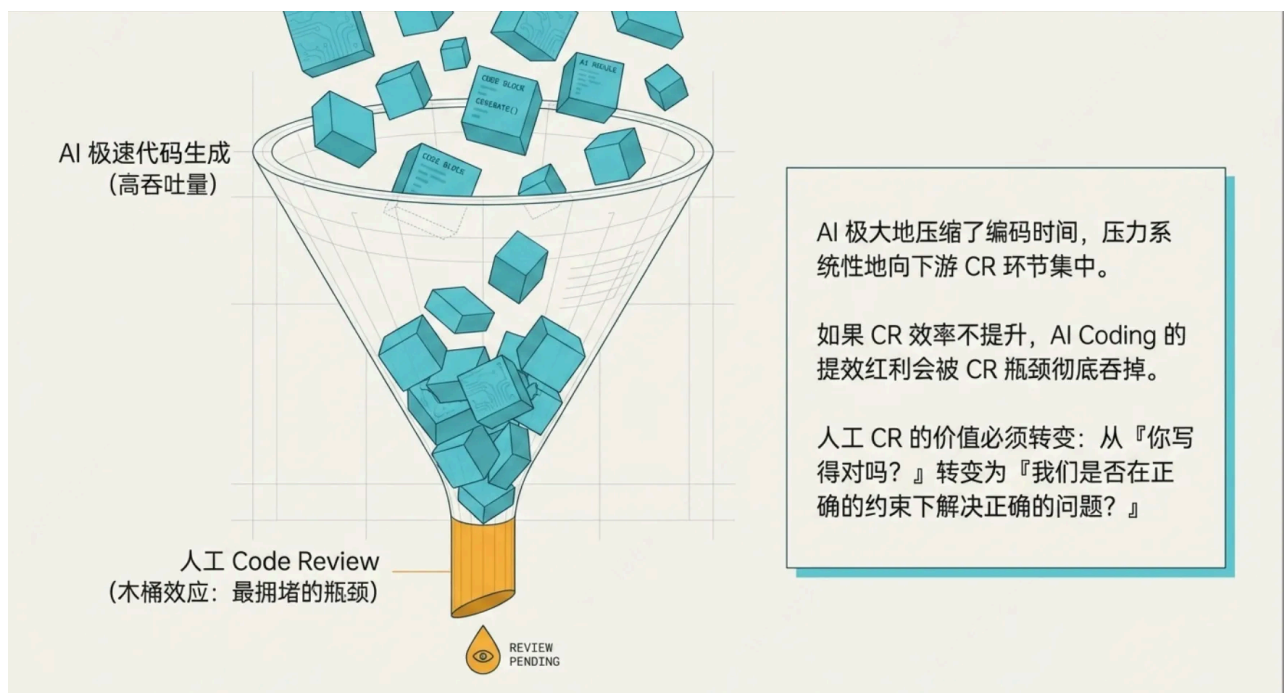
我们的实践经验：

1、引入 Pre-PR（预审）机制：

- 提交代码前，要求 RD 必须先用 AI 审查代码进行多轮自查，修复所有 AI 能发现的问题（规范类、Bug类、异常处理、一致性、可扩展性及性能问题等）。
- 确认通过后，提交标准的 PR 文档（重点说明改动点、影响范围、需重点 Review 的业务逻辑，AI 根据代码改动按模板生成）。
- 这样 Reviewer 拿到的就是一份“已过滤掉基础规范错误”的高质量代码，只需聚焦核心业务语义，认知负担大幅降低。

2、高阶模型审查低阶模型：使用高配模型作为 Judge Model，审查低阶模型产生的编码。

3、不同厂商模型对抗互相审核：使用不同厂商的模型互相审查对方的编码产出，通过差异化的模型能力形成互补，实测下来 CR 覆盖面更全。



2. 调研取经，建立AI 辅助测试用例生成规范

我们团队 100% 的需求由研发兼任测试（RD as QA）。在探索 AI 辅助自测时，团队自然演化出两条路线：路线 A 让 AI 全自动生成用例，人只做最后把关；路线 B 由人界定测试范围和风险级别，AI 负责代码扫描和用例步骤填充。

实践下来，路线 A 很快暴露出严重的工程问题 —— AI 缺乏全局业务认知，极度依赖 PRD 质量，容易漏掉隐性关联的高危场景，同时发散出大量无价值的边缘用例，反而增加 Review 负担。与专业 QA 团队交流后，我们确认了路线 B（人工主导，AI 辅助）的方向，并沉淀为一套 Human-in-the-loop 的测试 SOP：

步骤	目标	人做什么	AI做什么	AI提效点
Step1 建立范围	确定要测哪些接口	审核并确认最终测试范围，排除误报	从流量监控+代码变更双向扫描，自动收集受影响接口清单	省去人工逐个翻代码找接口的时间，避免遗漏
Step2 风险分级	决定每个接口测多深	根据 AI 提供的信息判定风险等级，决定测试深度	读代码回答三个问题：改了多少、分支在哪、旧数据是否兼容	把“读代码评估风险”从小时级压缩到分钟级
Step3 设计分组	最小化用例数量	审核分组合理性，补充业务特殊场景	用判定表方法“先拆后合”，自动生成最小Case组合	组合爆炸的场景下，AI比人算得快且不易出错
Step4 生成步骤	写出可执行的测试步骤	校验步骤是否匹配实际改动，补充边界条件	按“一步操作、多维验证”模板展开，匹配改动级别生成步骤	批量生成结构化用例。人只需 Review 不需从零写
Step5 验证覆盖	确保不漏不多	最终确认覆盖矩阵无盲区	自动生成接口×维度的覆盖矩阵，标记未覆盖项	交叉验证靠人肉极易遗漏，AI 做矩阵比对零成本

四、总结

我们沉淀的关键经验

- 用评测 Agent 的方式管理 AI Coding**：我们团队负责 Agent 评测业务，在实践中沉淀出“人人对齐→人机对齐”的核心理念，而管理 AI Coding 的底层逻辑与之完全相同。先让团队形成统一共识（人人对齐），再将共识固化为 AI 可执行的约束（人机对齐）。顺序错了，AI Rule 写得再好也是一纸空文。
- AI 重新定义了“经验”的价值边界**：利用 AI 工具，工程师短时间内就发现了 10 个性能隐患。过去要靠三年经验才能建立的代码全局感，AI 给所有人都配上了。经验的价值正在从“能看全”转移到“能判断什么重要”——这才是人不可替代的部分。
- 技术债可以像业务需求一样被迭代消化**：重构不需要排期，需要拆解能力。我们没有申请一天专门的重构时间，31 万行代码在业务交付中被渐进式消化。关键在于能不能把技术债拆解为业务需求的顺带动作。这需要极强的技术判断力，但一旦跑通，重构就不再是与业务争资源的零和博弈。
- 工程师的角色变了**：当 90% 代码由 AI 生成，团队成员的工作重心应从“写代码”转向“设计并维护一个能让 AI 可靠产出代码的工程环境”。

工程师角色的根本转变

在代码 90% 由大模型生成的时代，
研发团队的核心工作重心必须发生转移：
从单纯的『编写业务代码』，转向设计并维护一个
能让 AI 稳定、可靠产出代码的『工程环境』。

先人人对齐，
再人机对齐。

规范不仅是协作文档，
更是约束 AI 的基础设施。

行动指南：如果你的团队也想落地

- **第一步：**盘清技术债。不要试图人工遍历，由核心开发圈定高危方向，让 AI 做穷举扫描。投入相对较小，但能建立全局认知。
- **第二步：**制定规范并落地为 AI Rule、Skill。先组织团队对齐分层原则、建模方式、依赖边界等核心共识，再将共识固化为 AI 编码时 always 加载的 Rule。规范不落地到 AI 工具链里，就只是一纸空文。
- **第三步：**由主 R 打样，沉淀可复用的迁移 SOP。不要让每个人自己摸索，先由一个人跑通一个模块的完整迁移流程，把步骤沉淀为 AI 可执行的 SOP，再推广到全组。
- **第四步：**建立 Pre-PR 机制。要求每次提交前先用 AI 按团队规范自查，过滤掉基础问题，让人工 CR 只聚焦业务语义。AI 编码提速后，CR 会成为新瓶颈，这一步不能省。

用Agent评测思路管理AI Coding —— 31万行代码AI重构的实践 | 美团 · 技术团队

<https://tech.meituan.com/2026/05/07/Agent-AI-Coding.html>

